Acta del Encargado de Pruebas — Rutas del Bajío

Act02_P2 · Etapa 2 (equipo). Contiene la matriz de criterios de prueba (Tabla 2) y las decisiones de estructura justificadas (Tabla 3). Es el plano de la suite que se implementará a continuación.

Equipo	Eugenio Team (Prof. Eugenio Salvador Martínez Velázquez)
Fecha	14 de octubre de 2026
Encargado de Pruebas	Edgar Alexis Sánchez Huerta
Arquitecto	Edgar Daniel Zuñiga Gonzalez
Desarrollador CRUD	Cristian Ibrahim Salomon Marquez
Documentador	Luis Mario Juarez Rolón

Sección A. Alcance: componentes que se probarán

Código bajo prueba: `Desarrollo.ipynb` (cuaderno del equipo). Para poder importarlo desde `pytest`, sus celdas finales de clases se copian sin cambios a un paquete (ver Tabla 3, decisión 1).

Componente (clase o método)	Qué hace	Origen
Validadores: `texto`, `clave`, `validar_ciudad`, `entero_positivo`, `decimal_positivo`, `fecha_iso`	Normalizan y rechazan datos inválidos antes de crear una entidad	Sprint 1
`Cliente`, `Ruta`, `Pedido` (`__init__`, `to_dict`, `from_dict`)	Entidades con validación y conversión a diccionario/CSV	Sprint 1
`GestorRutasBajio.cargar` / `guardar` / `_leer` / `_escribir`	Persistencia en CSV (clientes, rutas, pedidos, secuencias)	Sprint 1
`crear_cliente`, `crear_ruta`, `crear_pedido`, `registrar_pedido`, `buscar_*`	CREATE y búsqueda; rechazo de ids duplicados; folio automático	Sprint 1
`listar_*`, `pedidos_por_cliente`, `pendientes_por_transportista`, `rutas_compatibles`, `detalle_pedido`	READ y consultas a través de las relaciones	Sprint 2
`actualizar_cliente` / `_ruta` / `_pedido`, `asignar_ruta`	UPDATE con revalidación de relaciones	Sprint 1
`_validar_relaciones`	Composición: cliente existente, ruta existente, mismo trayecto, ruta disponible	Sprint 2
`cambiar_estado`, `cancelar_pedido`	Máquina de estados del pedido	Sprint 2
`eliminar_cliente` / `_ruta` / `_pedido`	DELETE con protección de integridad referencial	Sprint 2

Nota: en la columna *Origen* se asignó "Sprint 1 = entidades y CRUD básico", "Sprint 2 = relaciones (composición) y estados".

Fuera de alcance: capacidad de camiones (la disponibilidad es manual, según el análisis del caso) y la ejecución en Colab.

Tabla 1. Reglas y riesgos por componente

Componente	Regla de negocio o comportamiento esperado	Riesgo: qué podría salir mal	Pregunta detonadora
`clave`	Las claves siguen `CLI-NNN`, `RUTA-NN`, `PED-NNNN` (se pasan a mayúsculas)	Aceptar una clave mal formada y romper búsquedas y relaciones	Entradas inválidas
`Cliente.__init__`	Teléfono con 10 a 15 dígitos; ciudad dentro del catálogo	Teléfonos de 9 o 16 dígitos aceptados; ciudades inventadas	Límites
`entero_positivo`	La cantidad es un entero mayor que cero	Aceptar 0, negativos, `True` o texto	Entradas inválidas

Componente	Regla de negocio o comportamiento esperado	Riesgo: qué podría salir mal	Pregunta detonadora
Ruta / Pedido.__init__	Origen y destino distintos	Pedido de León a León	Entradas inválidas
Pedido.__init__	En tránsito o entregado exige ruta	Pedido que sale sin transportista	Relaciones
fecha_iso	Formato AAAA-MM-DD y fecha real	2026-02-30 aceptada	Entradas inválidas
to_dict / from_dict	Ida y vuelta idéntica (None↔"", bool, float)	Al recargar cambia un tipo o se pierde la ruta	Persistencia
cargar	Sin archivos → listas vacías y ultimo_folio == 0	Excepción en el primer arranque	Archivo CSV
_leer	Encabezados exactos y filas completas	Cargar datos corruptos en silencio	Archivo CSV
_escribir	Escritura por archivo temporal que reemplaza al original	Archivo a medias o .tmp huérfano	Persistencia
CSV con comas y acentos	"Aceros, Fundición del Bajío" vuelve idéntico	Columnas desplazadas	Archivo CSV
crear_*	Id repetido se rechaza sin modificar el archivo	Registro duplicado o archivo alterado	Límites
_validar_relaciones	Cliente y ruta existen, mismo trayecto, ruta disponible	Pedido huérfano o asignado a una ruta que no hace ese trayecto	Relaciones
cargar (pedidos)	Pedido con cliente o ruta inexistente → error	Llave foránea rota en el CSV pasa inadvertida	Relaciones
registrar_pedido / eliminar_pedido	Folio consecutivo que no se reutiliza tras una baja	Dos pedidos con el mismo folio a lo largo del tiempo	Persistencia
cambiar_estado	Solo pendiente→en_tránsito/cancelado y en_tránsito→entregado	Reabrir un entregado; cancelar tras salir	Entradas inválidas
eliminar_cliente / _ruta	No se eliminan si tienen pedidos	Pedidos huérfanos	Relaciones
pendientes_por_transportista, rutas_compatibles	Solo pedidos pendientes/en tránsito; solo rutas disponibles del mismo trayecto	Reporte con pedidos entregados o rutas ocupadas	Caso normal

Tabla 2. Matriz de criterios de prueba

Tipos: normal · límite · error · persistencia · relación. Prioridad: A (alta), M (media), B (baja), justificada por el daño al proceso de negocio.

ID	Componente	Criterio (qué se verifica)	Tipo	Escenario y datos	Resultado esperado	Prior.	Cómo se verificará
C-01	clave / Cliente.__init__	Rechaza claves mal formadas	Error	"CLI-1", "", "PED-0001", "cli" como id_cliente	ValueError en cada caso	M — el error se detecta al crear, sin afectar datos guardados	parametrize + pytest.raises
C-02	Cliente.__init__	Teléfono dentro de 10 a 15 dígitos	Límite	9, 10, 15 y 16 dígitos	9 y 16 → ValueError; 10 y 15 se aceptan	M — dato de contacto, no rompe el proceso	parametrize + pytest.raises
C-03	validar_ciudad	Normaliza mayúsculas y rechaza ciudades fuera del catálogo	Error	"león", "CELAYA" y "Morelia"	"León", "Celaya"; "Morelia" → ValueError	M — evita rutas hacia ciudades que la empresa no atiende	parametrize + pytest.raises
C-04	entero_positivo / Pedido.__init__	Cantidad debe ser entero mayor que cero	Error	0, -5, "muchas", "3.5", True; y "40" válido	ValueError en los inválidos; "40" → 40	A — una cantidad errónea distorsiona la carga y la facturación	parametrize + pytest.raises
C-05	Ruta / Pedido.__init__	Origen y destino deben ser distintos	Error	Origen y destino "León"	ValueError en ambas clases	M	pytest.raises
C-06	Pedido.__init__	En tránsito o entregado exige ruta	Error	estado="en_transito", id_ruta=None	ValueError	A — un pedido sin transportista no se puede rastrear	pytest.raises
C-07	fecha_iso	Fecha con formato y calendario válidos	Error	"2026-02-30", "15/09/2026", ""	ValueError en cada caso	M	parametrize + pytest.raises

ID	Componente	Criterio (qué se verifica)	Tipo	Escenario y datos	Resultado esperado	Prior.	Cómo se verificará
C-08	to_dict / from_dict	Ida y vuelta conserva tipos y valores	Persistencia	Pedido sin ruta y con RUTA-03; Ruta con disponible=False	from_dict(to_dict(x)).to_dict() == x.to_dict()	A — si se pierde un campo, se pierde el dato al recargar	assert
C-09	GestorRutasBajio.cargar	Arranca vacío si no existen los CSV	Límite	Directorio temporal sin archivos	Listas vacías, ultimo_folio == 0, sin excepción	A — primer arranque del sistema	assert
C-10	GestorRutasBajio.cargar	Un CSV con solo encabezado equivale a vacío	Límite	clientes.csv con una línea de encabezado	clientes == [], sin excepción	M	assert (CSV escrito en la prueba)
C-11	crear_* + cargar	Lo guardado vuelve igual y en el mismo orden	Persistencia	Crear CLI-010, RUTA-01 y PED-0001, PED-0002; instanciar otro gestor sobre el mismo directorio	Mismos registros y mismo orden	A — dato perdido es el mayor daño al negocio	Prueba de integración + assert
C-12	_leer	Rechaza encabezados incorrectos y filas incompletas	Error	clientes.csv con encabezado id,nombre; fila con menos columnas	ValueError ("Encabezados incorrectos" / "Fila incompleta")	M — evita cargar datos corruptos como si fueran válidos	pytest.raises + CSV escrito en la prueba
C-13	guardar / _leer	Comas y acentos sobreviven al CSV	Persistencia	Cliente "Aceros, Fundición del Bajío", ciudad "León"	Mismo nombre al recargar; mismas columnas	M	assert tras recargar
C-14	_escribir	No deja archivo .tmp tras guardar	Persistencia	Crear un cliente	No existe ningún *.tmp en el directorio	B — residuo cosmético, el dato sí queda guardado	assert sobre tmp_path.glob("*.tmp")
C-15	crear_cliente	Rechaza ids duplicados sin tocar el archivo	Error	Dos clientes CLI-010	ValueError; clientes.csv idéntico antes y después	A — id duplicado	pytest.raises + assert
C-16	crear_pedido	Cliente inexistente	Relación	Pedido de CLI-404	ValueError; pedidos == [] y archivo sin cambios	A — pedido huérfano	pytest.raises + assert
C-17	crear_pedido / asignar_ruta	La ruta debe hacer el mismo trayecto del pedido	Relación	Pedido León→Irapuato con RUTA-03 (León→Guanajuato Capital)	ValueError; el pedido no se asigna	A — carga enviada a un destino equivocado	pytest.raises
C-18	crear_pedido	No asigna rutas no disponibles	Relación	RUTA-01 con disponible=False	ValueError ("no está disponible")	M — la disponibilidad es manual, el riesgo es operativo	pytest.raises
C-19	cargar	Pedido huérfano en el CSV impide la carga	Relación	pedidos.csv con id_cliente=CLI-099 inexistente	ValueError ("El cliente de PED-… no existe")	A — llave foránea rota	pytest.raises + CSV escrito en la prueba
C-20	eliminar_cliente / eliminar_ruta	No elimina si hay pedidos asociados	Relación	CLI-010 y RUTA-03 con PED-0001	ValueError; el registro sigue existiendo tras recargar	A — pedidos huérfanos	pytest.raises + assert tras recargar
C-21	registrar_pedido	Genera folios consecutivos	Normal	Dos registros con ultimo_folio == 0	PED-0001 y PED-0002	A — el folio identifica al pedido	assert
C-22	eliminar_pedido + cargar	El folio no se reutiliza tras una baja	Persistencia	Registrar PED-0001, eliminarlo, recargar y registrar otro	El nuevo es PED-0002; ultimo_folio == 1 tras recargar	A — dos pedidos con el mismo folio	Prueba de integración + assert
C-23	cambiar_estado	Transiciones válidas	Normal	pendiente→en_tránsito→entregado con ruta disponible	True en cada paso; estado persiste al recargar	A	assert
C-24	cambiar_estado / cancelar_pedido	Transiciones prohibidas	Error	entregado→pendiente, en_tránsito→cancelado, pendiente→entregado	ValueError ("Cambio no permitido"); estado intacto	A — reabrir entregados o cancelar tras salir	parametrize + pytest.raises
C-25	eliminar_pedido	Solo baja de pendientes o cancelados; id inexistente devuelve False	Error	PED-0001 en tránsito; PED-9999	ValueError; False	M	pytest.raises + assert

ID	Componente	Criterio (qué se verifica)	Tipo	Escenario y datos	Resultado esperado	Prior.	Cómo se verificará
C-26	pendientes_por_transportista	Solo cuenta pendientes asignados y en tránsito; incluye transportistas sin pedidos	Normal	Pedidos pendiente, en tránsito, entregado y cancelado	Entregado/cancelado excluidos; transportista sin pedidos → []	M	assert
C-27	rutas_compatibles	Devuelve solo rutas disponibles con el mismo trayecto	Normal	3 rutas: una distinta, una no disponible, una válida	Lista con solo la válida	M	assert

Cobertura de mínimos: 27 criterios (mín. 12) · error: 10 (mín. 3) · persistencia: 5 (mín. 2) · relación: 5 (mín. 2) · límite: 3 (mín. 1) · normal: 4 · prioridad A: 14.

Tabla 3. Decisiones de estructura de la suite

Decisión	Alternativas consideradas	Decisión tomada	Justificación	Responsable
1. Archivos de prueba y nombres (y dónde vive el código)	a) Un archivo por clase; b) un archivo por sprint; c) un archivo por área de comportamiento. Para el código: probar desde el .ipynb o exportarlo a un paquete	(c): test_validadores.py, test_modelos.py, test_csv.py, test_gestor_crud.py, test_relaciones.py, test_estados.py, test_integracion.py. El código de las celdas se copia sin cambios a rutas_bajio/modelos.py y rutas_bajio/gestor.py	Solo hay tres entidades y un gestor (GestorRutasBajio): "uno por clase" dejaría un archivo enorme para el gestor. Por sprint se mezclarían temas. Un cuaderno no se importa desde pytest, por eso se exporta	Alexis
2. Fixtures y conftest.py	a) Construir el gestor dentro de cada prueba; b) fixtures en conftest.py	(b): directorio (tmp_path / "datos"), gestor (vacío), cliente, ruta, gestor_con_datos (CLI-010, RUTA-01/RUTA-03, un pedido pendiente)	Evita repetir la preparación y deja las pruebas cortas. El gestor usa por defecto datos_notebook/; las fixtures siempre le pasan tmp_path para no tocar datos reales	Daniel
3. Datos de prueba	a) CSV de ejemplo versionados en el repo; b) tmp_path con CSV escritos dentro de la prueba	(b)	Cada prueba es aislada y repetible, sin depender de archivos compartidos ni del orden. Los CSV rotos (C-10, C-12, C-19) se ven en la propia prueba. Mínimo de registros: 2 clientes, 3 rutas (una no disponible) y 2 pedidos	Cristian
4. Parametrización	a) Una prueba por valor inválido; b) parametrize con pytest.param(..., id=)	(b) en C-01, C-02, C-03, C-04, C-07 y C-24	Mismas reglas con distintos datos; el reporte muestra cada caso por nombre ([telefono_9_digitos]) y se evita copiar y pegar	Cristian
5. Marcadores y ejecución por partes	a) Sin marcadores; b) integracion solamente; c) integracion y csv	(c), registrados en pytest.ini con --strict-markers. integracion: C-11, C-22, C-23. csv: C-09 a C-14, C-19	Las unitarias se corren siempre (-m "not integracion"); las de integración y las de archivos, antes de entregar. --strict-markers evita marcadores mal escritos que dejarían pruebas sin ejecutar	Daniel
6. Orden de implementación y reparto	a) Por orden de ID; b) por prioridad; c) por integrante sin orden	(b): primero Alta (C-04, 06, 08, 09, 11, 15–17, 19–24), luego Media y Baja. Reparto por integrante (ver abajo)	Lo que más dañaría al negocio (dato perdido, relación rota, id duplicado, estado ilegal) queda protegido desde el principio. Al cerrar, se aplica la técnica de mutación del Paso 10 (p. ej. quitar la validación de duplicados) para comprobar que la suite las detecta	Alexis
7. Convención de nombres	a) test_1, test_2; b) test_<componente>_<escenario>; c) nombres con el ID del criterio	(b) con el ID en el docstring: test_crear_cliente_duplicado_lanza_value_error ("""C-15""")	El reporte se lee solo; el docstring permite rastrear cada prueba hasta su criterio de la Tabla 2	Luis Mario

Árbol de archivos propuesto

```
Eugenio-Team/
├── pytest.ini              <- testpaths = tests · pythonpath = . · addopts = -ra --strict-markers
│                           markers: integracion, csv
├── Desarrollo.ipynb        <- código original (no se prueba directamente)
├── rutas_bajio/            <- código bajo prueba, copiado sin cambios del cuaderno
│   ├── __init__.py
│   ├── modelos.py          <- validadores, Cliente, Ruta, Pedido
│   └── gestor.py           <- GestorRutasBajio
└── tests/
    ├── conftest.py         <- fixtures: directorio, gestor, cliente, ruta, gestor_con_datos
```

```
├── test_validadores.py     <- C-03, C-04, C-07
├── test_modelos.py         <- C-01, C-02, C-05, C-06, C-08
├── test_csv.py             <- C-09, C-10, C-12, C-13, C-14      (csv)
├── test_gestor_crud.py     <- C-15, C-21, C-25, C-26, C-27
├── test_relaciones.py      <- C-16, C-17, C-18, C-19, C-20
├── test_estados.py         <- C-24
└── test_integracion.py     <- C-11, C-22, C-23                  (integracion)
```

Reparto de las pruebas y revisión final

Integrante	Criterios (IDs) que implementará	Firma de revisión (nombre)
Edgar Alexis Sánchez Huerta (Encargado de Pruebas)	C-01, C-02, C-03, C-04, C-07, `conftest.py` y `pytest.ini`	
Edgar Daniel Zuñiga Gonzalez (Arquitecto)	C-16, C-17, C-18, C-19, C-20, C-23	
Cristian Ibrahim Salomon Marquez (Desarrollador CRUD)	C-09, C-10, C-11, C-12, C-15, C-21, C-22, C-25	
Luis Mario Juarez Rolón (Documentador)	C-05, C-06, C-08, C-13, C-14, C-24, C-26, C-27	

Lista de verificación (Paso 5)

☐ Cada criterio describe un comportamiento, no una implementación.
☐ Cada criterio tiene escenario concreto y resultado esperado verificable (se puede escribir un `assert`).
☐ La matriz cumple los mínimos: 12 criterios, 3 de error, 2 de persistencia, 2 de relación y 1 límite.
☐ Ningún criterio depende del orden de ejecución ni de archivos reales (todo usa `tmp_path`).
☐ Las prioridades Alta cubren dato perdido, relación rota e id duplicado.
☐ Cada decisión de estructura tiene alternativa descartada y justificación.
☐ Cualquier integrante puede explicar cualquier criterio del acta.